BurstGPT: A Real-World Workload Dataset to Optimize LLM Serving Systems

Yuxin Wang*
yuxin.wang11@huawei.com
Huawei Hong Kong Research Center
Hong Kong, CHINA

Yuhan Chen*
y.chen@connect.hkust-gz.edu.cn
The Hong Kong University of Science and Technology (Guangzhou)
Guangzhou, Guangdong, CHINA

Zeyu Li
zli755@connect.hkust-gz.edu.cn
The Hong Kong University of Science and Technology (Guangzhou)
Guangzhou, Guangdong, CHINA

Xueze Kang
xkang507@connect.hkust-gz.edu.cn
The Hong Kong University of Science and Technology (Guangzhou)
Guangzhou, Guangdong, CHINA

Yuchu Fang
fangyuchu96@huawei.com
Huawei Technologies Co.
Shenzhen, Guangdong, CHINA

Yeju Zhou
zhouyeju@huawei.com
Huawei Technologies Co.
Shenzhen, Guangdong, CHINA

Yang Zheng
zhengyang31@huawei.com
Huawei Technologies Co.
Shenzhen, Guangdong, CHINA

Zhenheng Tang
zhtang.ml@ust.hk
The Hong Kong University of Science and Technology
Hong Kong, CHINA

Xin He
csxinhe@comp.hkbu.edu.hk
Hong Kong Baptist University
Hong Kong, CHINA

Rui Guo
greatfeel@gmail.com
Tsinghua University
Beijing, CHINA

Xin Wang
wangxin@aichoice.cn
Tsinghua University
Beijing, CHINA

Qiang Wang
qiang.wang@hit.edu.cn
Harbin Institute of Technology (Shenzhen)
Shenzhen, Guangdong, CHINA

Amelie Chi Zhou
amelieczhou@comp.hkbu.edu.hk
Hong Kong Baptist University
Hong Kong, CHINA

Xiaowen Chu†
xwchu@hkust-gz.edu.cn
The Hong Kong University of Science and Technology (Guangzhou)
Guangzhou, Guangdong, CHINA

Abstract

Despite efforts to improve the quality of service (QoS) and throughput in Large Language Model (LLM) serving systems, progress is often limited by the lack of publicly available real-world workloads. Consequently, evaluations usually depend on synthetic or oversimplified load patterns, and systems that appear promising in testing frequently underperform once deployed.

This work presents `BurstGPT`, an LLM serving workload with 10.31 million traces from regional Azure OpenAI GPT services over 213 days. `BurstGPT` captures LLM serving characteristics from user, model and system perspectives: (1) User request concurrency: burstiness variations of requests in Azure OpenAI GPT services, revealing diversified concurrency patterns in different services and model types. (2) User conversation patterns: counts and intervals within conversations for service optimizations. (3) Model response lengths: auto-regressive serving processes of GPT models, showing statistical relations between requests and their responses. (4) System response failures: failures of conversation and API services, showing intensive resource needs and limited availability of LLM services in Azure. The details of the characteristics can serve multiple purposes in LLM serving optimizations, such as system evaluation and trace provisioning. In our demo evaluation with `BurstGPT`, frequent variations in `BurstGPT` reveal declines in efficiency, stability, or reliability in realistic LLM serving. We identify that the generalization of KV cache management, scheduling and disaggregation optimizations can be improved under realistic workload evaluations. `BurstGPT` is publicly available now at https://github.com/HPMLL/BurstGPT and is widely used to develop prototypes of LLM serving frameworks in the industry.

*Both authors contributed equally to this research.
†Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

KDD '25, Toronto, ON, Canada

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1454-2/2025/08
https://doi.org/10.1145/3711896.3737413

CCS Concepts

• **Computing methodologies → Modeling and simulation**.

Keywords

LLM Serving, Workload Trace, Workload Management, System Scheduling

ACM Reference Format:

Yuxin Wang, Yuhan Chen, Zeyu Li, Xueze Kang, Yuchu Fang, Yeju Zhou, Yang Zheng, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. 2025. BurstGPT: A Real-World Workload Dataset to Optimize LLM Serving Systems. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '25), August 3–7, 2025, Toronto, ON, Canada.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3711896.3737413

1 Introduction

Large language models (LLMs), particularly Generative Pre-trained Transformer (GPT) models, have significantly impacted the AI industry. Notable examples include OpenAI's ChatGPT [29, 30], Google's Gemini [41], and Meta's Llama [42]. However, deploying and operating LLM services is costly due to the substantial use of AI accelerators [21]. The high volume of user requests and limited hardware resources further constrain the quality of service (QoS) [21]. Therefore, optimizing serving systems to reduce costs and improve user experience is urgent.

Existing systems primarily optimize system throughput and achieve better QoS. However, these studies often use non-LLM serving workloads in evaluations, derived from synthetic models [43] or non-LLM services such as Microsoft Azure Functions (MAF) [36, 50]. This work identifies that these unrealistic synthetic workloads do not reflect the workload patterns of auto-regressive LLM serving and limit the effectiveness of evaluating these systems.

In addressing this gap, we introduce a real-world LLM serving workload dataset named `BurstGPT`. As shown in Figure 1, the traces are collected from the Azure OpenAI GPT Service [26], which serves GPT models on Azure and provides APIs for regional GPT service providers, such as enterprises and campuses, to build customized GPT services. We collected data from one of these regional GPT service providers with over 3,000 users. By deploying a logging engine on this regional service provider, we monitored privacy-independent metrics of each GPT request. In Figure 1, we collected the *request start time*, *request and response token lengths*, *service type* (API or conversational), and *LLM type* (ChatGPT, GPT-4 or GPT-4v) for 213 days. The dataset comprises 8.69 million traces of ChatGPT and 0.95 million GPT-4 traces using API services, and 0.30 million ChatGPT traces and 0.16 million traces of GPT-4 using conversational services. It also includes failure requests while the response length is zero.

To use `BurstGPT`, the user can sample a period of trace and scale it to fit the target system size. Users can apply vanilla `BurstGPT` using request-per-second (RPS) scaling or model the trace and adjust parameters, as shown in Figure 1. The subset can be used for various optimizations, such as identifying concurrency patterns for provisioning and predicting response lengths based on request-response length distributions. Additionally, this paper provides a detailed

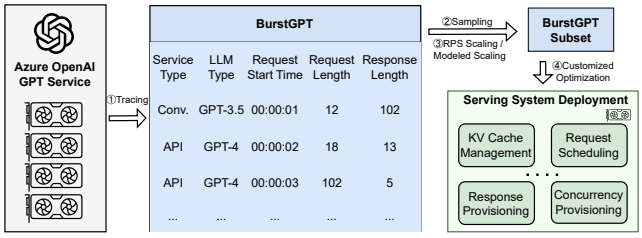


Figure 1: Data collection and use method of `BurstGPT`. `BurstGPT` is a real-world workload trace from the Azure OpenAI GPT service. A scaled sample from a period of `BurstGPT` can be used to optimize serving systems using specific methods, considering realistic concurrency and response patterns. Note that we open-sourced two versions of `BurstGPT`: a cleaned trace and a raw trace, with failure logs excluded from the cleaned version.

demo of `BurstGPT` for LLM serving evaluation. Key findings from our analysis and evaluations are listed below:

- User Request Concurrency. We identify unique patterns in request concurrency and lengths, varying by service types. This insight suggests that service-specific user request pattern evaluations are important for optimizing LLM serving. In our evaluation, (1) concurrency from non-LLM workloads does not accurately reflect system performance as `BurstGPT` does. (2) Even for LLM workloads, generalization of system optimization across serving types is not guaranteed. For example, we verify that while optimized first-come-first-served scheduling is effective for conversation services, its efficiency can be reduced for API services.
- User Conversation Patterns. We share traces for the first 2 months in 3.1, 3.3, 3.4, and conversation traces from the last 3 months in 3.2. The detailed analysis of conversation traces can help schedule LLM serving, e.g., KV cache offload strategies.
- Model Response Lengths. Longer response lengths lead to higher workloads during LLM serving. The unpredictable nature of response lengths per request introduces uncertainty to system pressure. The statistical study of the relationship between request and response lengths in `BurstGPT` sheds light on future response provisioning, which can help improve system performance.
- System Response Failures. We observe a relatively high failure rate in `BurstGPT`. In our evaluation, we identify that this is primarily due to inefficiencies in KV cache management. Variations in burstiness in `BurstGPT` lead to memory bottlenecks, causing spikes in failure rates and performance degradation.

With the dataset, we open-sourced a benchmark suite, BurstGPT-Perf. In our evaluation using BurstGPT-Perf, we first present our insights on using `BurstGPT` to assess system performance differences between LLM and non-LLM workloads. We also demonstrate use cases of `BurstGPT` in scheduling policy selection, workload

provisioning. Then, we share our insights in prefill-decode(PD) disaggregation in industrial system prototypes. The evaluation results show that BurstGPT can effectively suggest optimizations for LLM serving systems in real-world scenarios.

2 Preliminary and Motivation

Table 1: Comparison of BurstGPT with Other Traces

Serving Workloads	Real-world			Synthetic
	BurstGPT	MAF1 [36]	MAF2 [50]	FastServe [43]
LLM-User Req. Pat.	✓	×	×	×
LLM-User Conv. Pat.	✓	×	×	×
LLM Resp. Len.	✓	×	×	✓
Sys. Resp. Fail.	✓	×	×	×

2.1 Limitations of LLM Serving

Each request in LLM serving requires uncertain but large computational resources due to the model size and auto-regressive nature [17]. To reduce deployment and operational expenses, several specialized frameworks have been developed, including TensorRT-LLM [27], vLLM [17], DeepSpeed [34], and lightLLM [11]. In addition to these frameworks, various memory and compute optimizations have been proposed to improve serving efficiency [3, 5–8, 10, 13, 14, 32, 35, 37, 40, 47, 54], significantly improving LLM inference performance in deployments. Also, efficient request scheduling is crucial for optimizing LLM service. For example, Orca [49] introduces iteration-level scheduling, which dynamically adjusts batch size during iterations; FastServe [43] proposes optimized Multilevel-Feedback-Queue (MLFQ) scheduling in the request queue to improve system efficiency. Sarathi-Serve [1] utilizes chunked prefill to prevent long prefill tasks from stalling the decoding tasks, thereby enhancing overall hardware utilization.

However, these optimizations have yet to be evaluated on real-world LLM serving workloads. We identify that generalizing serving optimizations from non-LLM or synthetic workloads to LLM workloads can be challenging for ensuring serving efficiency [24, 44–46, 51], stability [25, 38, 39] and reliability [9, 18, 22, 48].

2.2 Towards Workload-aware LLM Serving

Stable workloads lead to consistent system performance across various metrics. However, in the real world, the workload of LLM is diversified due to user, system, and model behaviors. Neglecting any aspect of workload traces during system optimization may yield an incomplete understanding of a framework's behavior in real-world deployment.

User Concurrency Patterns. Currently, the assessment of performance in LLM serving frameworks relies on synthetic concurrency [43] or non-LLM concurrency, such as Microsoft Azure Functions (MAF) [36, 50]. As concluded in Table 1, these approaches do not reflect real-world LLM workloads due to a lack of empirical data on system and model patterns, as well as user behaviors. For example, the request-per-second (RPS) metric in MAF is much higher (1.64 RPS on average) compared to LLM services (0.019 RPS on average in conversation service and 0.21 RPS on average in API service when using ChatGPT) due to the non-autoregressive and lightweight nature of function services in Azure. However, burstiness in concurrency is much less severe in MAF, likely leading to performance divergence from LLM workloads.

Model Patterns. The computational complexities of prefilling and decoding in LLM systems are $O(s^2)$ and $O(s)$ relative to the request length s, respectively [16]. This relationship between request length and system load significantly differs from that observed in other lightweight cloud functions and synthetic workloads [36, 50], highlighting a unique resource occupation of LLM serving. These complexities are also reflected in the request concurrency, failure rates, and latency of each request in LLM serving.

3 Introduction to BurstGPT

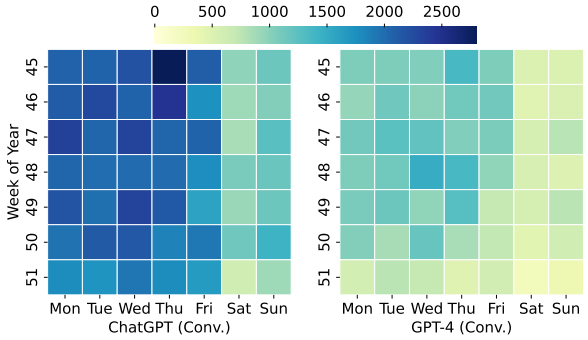


Figure 2: Weekly Periodicity of Conversation Services in BurstGPT.

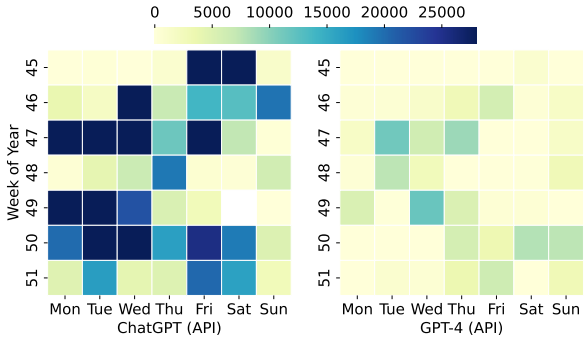


Figure 3: Weekly Aperiodicity API Services in BurstGPT.

3.1 User Request Concurrency

The long-term usage patterns of conversation services, spanning hours and days, exhibit periodic characteristics. As illustrated in Figure 2, conversation volumes for both ChatGPT and GPT-4 peak during weekdays and diminish during weekends, implying heightened user interaction primarily on workdays. Notably, GPT-4's overall usage is lower compared to ChatGPT. Figure 4 shows that conversation volumes for both ChatGPT and GPT-4 exhibit periodic highs during working hours and lows during night hours.

In contrast, the long-term pattern of API services, over the same hourly and daily intervals, follows an aperiodic pattern characterized by burstiness. Figures 3 and 5 reveal irregular, dense request submissions to API services, significantly diverging from the more

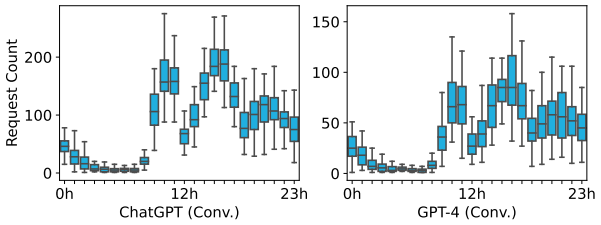


Figure 4: Daily Periodicity Conversation Services in BurstGPT.

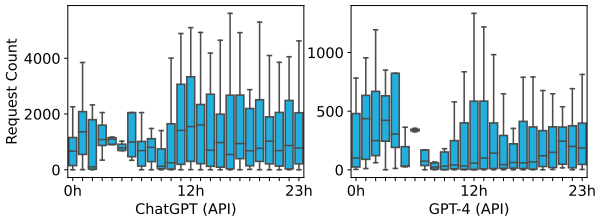


Figure 5: Daily Aperiodicity API Services in BurstGPT.

predictable patterns observed in conversation services. The higher variance in daily request volumes, indicated by a greater standard deviation compared to Figure 4, suggests these requests may be automated.

Short-term Patterns: Variant Burstiness. Figure 6 presents the modeled burstiness every twenty minutes in ChatGPT's conversation and API services during working hours on weekdays. The burstiness is modeled using the Gamma distribution, as supported in [21, 43]. We observe that the shape parameter α and the rate parameter β of the Gamma distribution vary between conversation and API services. A smaller α indicates a higher coefficient of variation (CV) of the Gamma distribution, meaning the workload is more bursty [43]. In conversation services, the parameter α varies sharply during working hours. In API services, both α and β vary sharply during the day. These observations underscore the instability of burstiness in the system.

3.2 User Conversation Patterns

Long-term Patterns: Periodicity and Aperiodicity.

Distribution. Over 35% of conversations end with only one request. The left of Figure 7 shows that the distribution of conversation requests exhibits an exponential decline, with a median of 2 and 75% of conversations with four or fewer requests.

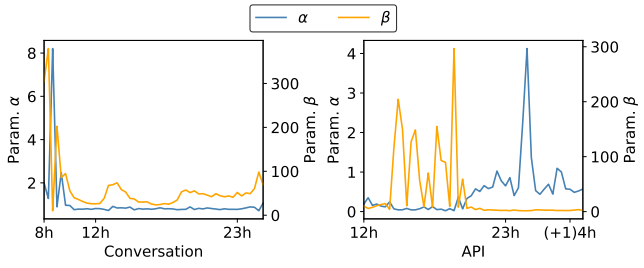


Figure 6: Variation of Burstiness in Gamma Distribution in BurstGPT.

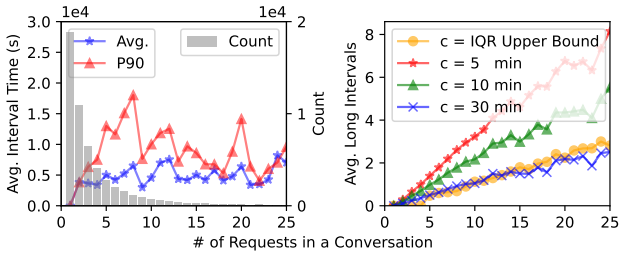


Figure 7: Count, Average Interval times, and Average Long Intervals as the Number of Requests in a Conversation in BurstGPT.

Interval Time. The left of Figure 7 shows that the average interval time is not directly related to the number of requests. The P90 interval time increases when the number of requests is less than 5 and then fluctuates after that.

The More Requests, the Longer Intervals. The right of Figure 7 illustrates the number of intervals exceeding a certain duration c in each conversation, which we define as long intervals. We make c equal to 3 different time deltas and the interquartile range (IQR) upper bound. These intervals increase as the number of conversation requests increases.

3.3 Model Response Patterns

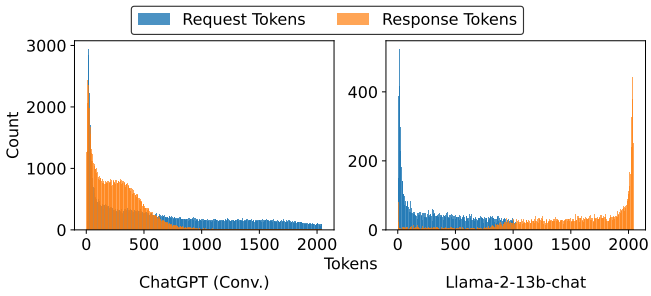


Figure 8: Distribution of Request and Response Tokens.

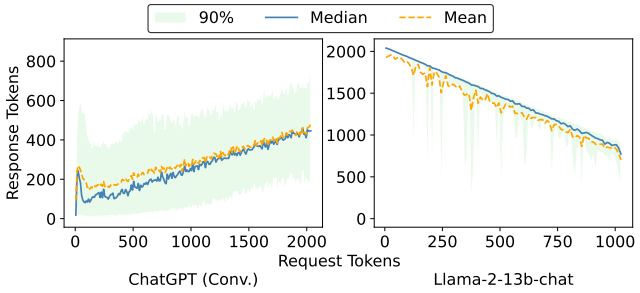


Figure 9: Response Lengths per Request

To identify differences in response length distributions across LLMs, this subsection employs conversational traces in BurstGPT and uses Llama-2-13b-chat[42] for comparison. For Llama-2-13b-chat, we feed 10k real-world prompts to the model for inference and gather the response lengths. The prompts are randomly truncated

from the ShareGPT dialogues [12] with aligned distribution to BurstGPT's conversational traces [52]. This alignment allows for comparing the response length distributions between the two LLMs, facilitating a deeper understanding of their behaviors in responding to conversational prompts.

Distribution of Request and Response Lengths. In Figure 8, we observe similarities in request distributions and differences in response distributions. Both ChatGPT and Llama-2-13b-chat request distributions adhere to a Zipf distribution [43], characterized by a peak in the frequency of shorter requests. The histogram of ChatGPT's response tokens reveals a bimodal distribution of length variability. In contrast, Llama-2-13b-chat exhibits a Zipf distribution with a higher frequency of longer requests. These distributions provide insights into the conversational dynamics of request and response lengths for the models. ChatGPT handles longer contexts compared to Llama, reflecting differences in model capacities or interactions.

Response Lengths per Request and Request Bin. ChatGPT shows a linear correlation between request and response lengths, with a symmetric distribution of response lengths in Figure 9. In contrast, Llama-2-13b-chat decreases response length as request length increases, with higher variability for longer requests due to context length limits.

In Figure 10, ChatGPT tends to produce shorter responses than Llama-2-13b-chat, indicated by a shifted Gaussian distribution with a peak at shorter lengths and a longer spread as request length increases. In contrast, Llama-2-13b-chat shows less variability in response length, peaking at 2048 characters.

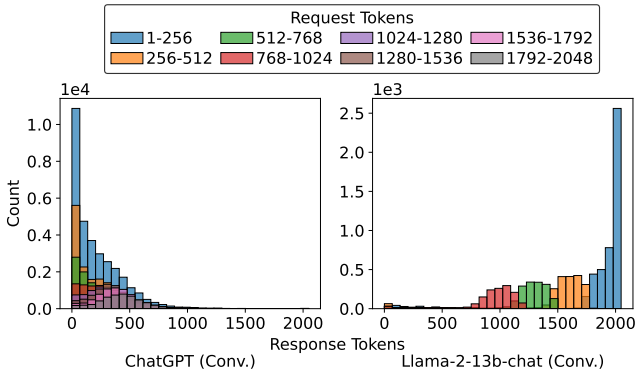


Figure 10: Response Lengths per Request Bin

3.4 System Response Failures

We observe a high failure rate in GPT services. Figure 11 shows that ChatGPT has a higher average service failure rate but better stability compared to GPT-4 in both conversational and API interfaces. Overall, GPT-4 has better service reliability over ChatGPT, demonstrating greater consistency and stability with fewer outliers. Additionally, the average failure rate of the conversation service is consistently high, exceeding 5% for ChatGPT, which is significantly higher than that of regular cloud services [2]. We will discuss the reliability problem in detail in Section 4.

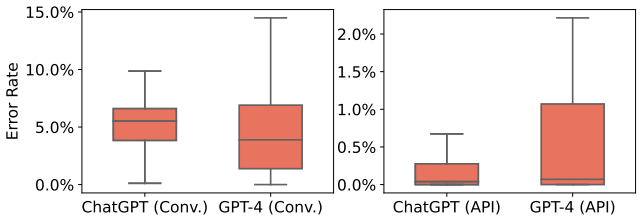


Figure 11: Comparison of Statistical Failure Rates in BurstGPT.

4 BurstGPT-Perf: A Benchmark Suite

This section presents our open-sourced benchmark suite: BurstGPT-Perf to evaluate LLM serving systems under BurstGPT with streaming, stochastic, and bursty workloads. The suite is lightweight and modular, requiring minimal code integration for effective deployment. Additionally, it serves as a user-friendly example of how to implement BurstGPT for evaluation and will be integrated and open-sourced within BurstGPT. With our use case, we advocate for using BurstGPT on more serving systems to explore system limitations and optimization opportunities further.

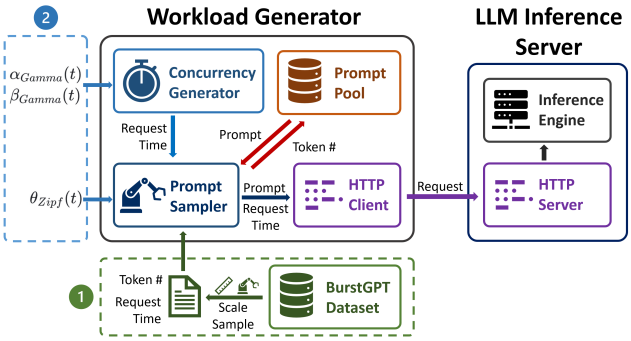


Figure 12: Overview of BurstGPT-Perf. It generates simulations of BurstGPT in a burst manner with two scaling methods: 1. *RPS Scaling* scales the original BurstGPT data; 2. *Modeled Scaling* uses Gamma distribution parameters to generate request times and Zipf distribution parameters to generate prompt token lengths.

4.1 Implementation

Workload Generator. Figure 12 presents an overview of the workload generator. The system comprises four main components: the `Prompt Sampler`, the `Prompt Pool`, the Concurrency Generator, and the `HTTP client`. The HTTP client is included using the efficient and asynchronous `aiohttp` framework. An index with token numbers as keys and prompt indices as values will be established in the `Prompt Pool` to mitigate the overhead of prompt sampler queries and a prompt is randomly selected for return. Once the prompt sampler retrieves a prompt from the `Prompt Pool` to enable workload concurrency, the concurrency generator times requests.

Evaluation Workflow. Requests of specified lengths are sampled from the `Prompt Pool` at stochastic intervals that are generated

by `Concurrency Generator` in the `Workload Generator`. This concurrency pattern is treated as a time series, which inputs it into the `Inference Engine` via `HTTP` for performance evaluation purposes. The benchmark suite systematically adjusts RPS if we use the vanilla trace, or the parameters λ, α, and β, according to a predetermined sequence, if we use modeled traces, thereby generating average and instantaneous performance metrics for users.

4.2 Scaling BurstGPT to Any Scale

In Section 3, we identify specific burstiness patterns from workload traces. To assess LLM serving systems on `BurstGPT` at any scale, we scale `BurstGPT` to match the system size, enabling scalable evaluations. We use *RPS Scaling* or *Modeled Scaling* in the evaluations. (1) For RPS scaling, we use a scaling parameter c to multiply the timestamp, making the average RPS in the scaled trace equal to the given RPS. (2) Modeled scaling offers more flexibility in adjusting the scaling parameters. Our analysis in Section 3 reveals that within `BurstGPT`, concurrency follows a varied Gamma distribution, characterized by the shape parameter α and the scale parameter β. The parameter α predominantly influences the coefficient of variation (CV) of the distribution, mathematically expressed as $CV = 1/\sqrt{\alpha}$.

To make `BurstGPT` applicable for evaluating LLM serving systems of different sizes, we adjust these parameters in `BurstGPT`'s concurrency distribution model at intervals of 20 minutes. For assessments of `BurstGPT` on a serving system of a particular scale, users can modify the β parameter to determine a warm-up arrival rate compatible with their system's capacity, such as 30% to 40% GPU utilization of KV cache in our setting. Following this adjustment, users can evaluate their system within any chosen time range in `BurstGPT`.

4.3 Matching BurstGPT with Dialogues

For request lengths, in Section 3 and related simulations [21], it is observed that the request length follows a Zipf distribution. The sampler samples request lengths using a variable parameter λ, influencing the frequency of shorter requests in input streams.

For response lengths in `BurstGPT`, they are not predefined, allowing the model to determine them dynamically. This strategy avoids restrictions that could degrade system runtime, especially with specific LLMs. Over many requests, the distribution of output lengths will naturally align with the model's behavior, as detailed in Section 3.

After determining request lengths, the suite initializes a CPU buffer for loading a truncated prompt dataset pool. This dataset can be sampled from existing GPT conversation dialogues, such as Alpaca [19]. The dataset truncation and request length sampling adhere to the predefined Zipf distribution. The buffer size is user-configurable to balance with the CPU's memory capacity. Requests of specific lengths are then selected from the prompt dataset and sent to the server at intervals, where they queue for processing.

4.4 Metrics and Setups

In serving systems, Quality of Service (*QoS*) is often measured by *latency* and *throughput* [23]. Latency refers to the completion time of each token, while throughput is the number of tokens processed within a given period. In addition to efficiency, the stability and reliability of LLM serving are particularly important to users, yet these aspects are often overlooked. In this evaluation, we also focus on latency jitters to assess stability and failure rates to assess the reliability of LLM serving.

We categorize metrics into two types: average values (denoted with a superscript ^{avg}) and instantaneous values (marked with a superscript ^{ins}). This categorization provides a comprehensive understanding of both long-term trends and immediate system behavior. Based on the trace study, the following key metrics are employed for a thorough evaluation of a serving system's performance:

- R^{avg} / R^{ins}: Request Failure Rate;
- L^{avg} / L^{ins}: Token Latency;
- σ_L^{avg} / σ_L^{ins}: Token Latency Jitters (Standard Deviation);
- P^{avg} / P^{ins}: System Throughput.

5 Demo Evaluations

Our evaluation is conducted on Llama-2-13b-chat on an A800 and A6000 GPU server. In experiments that use modeled scaling, we set the initial parameters α_{Gamma} and β_{Gamma} of the Gamma distribution in `BurstGPT` to 0.5 and 2, respectively, for 30% GPU KV Cache utilization. The request length parameter θ_{zipf} is set to 1.1. We use requests in ShareGPT as the request pool.

5.1 Evaluation: BurstGPT v.s. MAF2

In this evaluation, we assess the efficiency and stability of vLLM [16] using `BurstGPT` and MAF2. We set the same RPS to 0.326 to scale both MAF2 and `BurstGPT`. Both experiments were completed in 6126 seconds. Figure 13 shows the serving system's instantaneous metrics under the two workloads. The results indicate that the request burstiness of concurrency in `BurstGPT` is higher, whereas the request distribution of the MAF2 workload is more uniform. As we can see from Figure 14, the average latency and throughput are higher for `BurstGPT` compared to MAF2, indicating a larger system resource occupation by `BurstGPT`. In conclusion, although both methods share the same RPS, the more bursty concurrency in `BurstGPT` introduces more challenges to the serving system, particularly in terms of KV cache management, compared to MAF2.

5.2 Evaluation: Micro Workload Variations

In this experiment, we model the request lengths to a Zipf distribution, as in [20]. The sampler samples the request lengths in Zipf distribution with a variable parameter λ. We investigate the impact of varying specific variables: α, β, and λ. We utilized the Llama-2-7b-chat and Llama-2-13b-chat models in our testbed. The results demonstrate a correlation between the adjustments in these variables and the observed system behavior.

We observe that minor changes in the parameter α or the CV can lead to sudden request failures, significantly undermining the system's reliability. However, these changes have less effect on system performance metrics such as average latency and throughput. This is because such changes in α are temporary and behave as quadratic functions. In contrast, alterations in α in a linear decline lead to ongoing performance degradation, particularly noticeable during request failures. These findings underscore the value of our trace analysis and provide essential insights for improving system efficiency in similar computational environments.

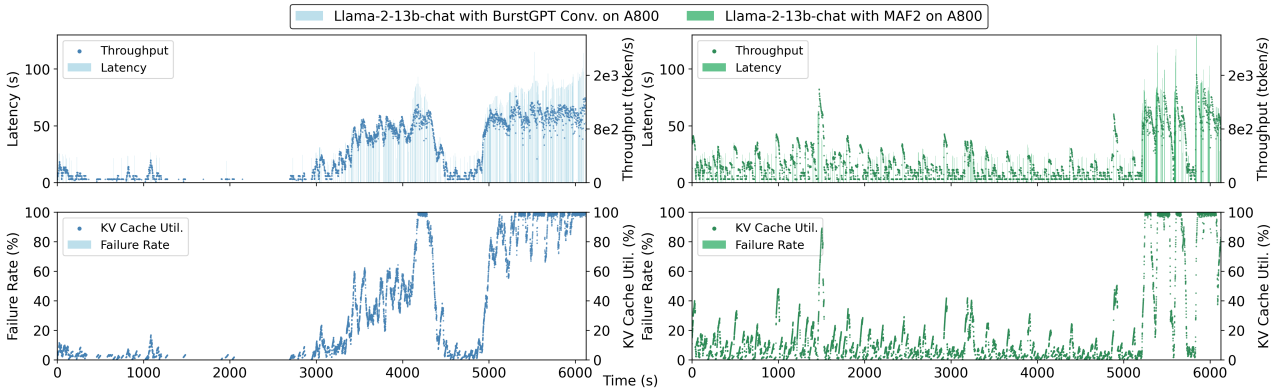


Figure 13: The impact of concurrency of `BurstGPT` Conversation and `BurstGPT` API traces on the latency, throughput, KV cache usage, and failure rates of vLLM serving on a single A800 GPU. Using the first 2000 queries for both traces, we scale BurstGPT by a factor of 10 to reach an RPS of 0.326. To reach the same RPS, MAF2 is scaled with a parameter of 1.234, ensuring both are complete in 6126 seconds.

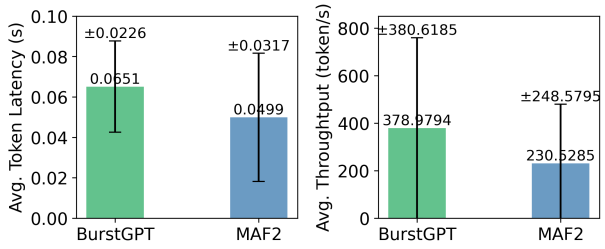


Figure 14: Average token latency and throughput of vLLM serving Llama-2-13b-chat on an A800 GPU server with `BurstGPT` and MAF2 workloads.

5.3 Demo Use: Request Scheduler Selection

We randomly sample a time range on `BurstGPT` and build two subsets with conversation workloads and API workloads, and then use Modeled Scaling to derive the corresponding parameter sequences for generating workloads. We use these two generated workloads separately to evaluate three scheduling strategies i.e. first-come-first-served (FCFS), short-request-first (SRF), and long-request-first (LRF), on vLLM.

In Figure 16, the efficiency of the various scheduling methods may shift across different types of real-world workloads. Comparing FCFS with SRF and LRF on average values in terms of efficiency and stability, we can find that the optimization of conversation service when LRF is superior to FCFS in latency and latency jitters does not hold true for API service. This interesting finding inspires us that even using real-world workloads, an optimization on one workload does not necessarily generalize to other workloads.

5.4 Demo Use: Workload Provisioning

To adjust the resources allocated to services promptly, cloud systems can predict system load to proactively scale resources up or down. The workload provisioning task is a time series forecasting problem. Specifically, we select the request count per time bin and the average number of tokens per request as the prediction targets. To construct features for the prediction model, we use historical load data from both short-term and long-term perspectives. For each time point, we incorporate load values from the past 3 time points (lag window) to capture short-term trends, along with statistical features (e.g., mean, variance) computed over a rolling window of up to 60 minutes to reflect long-term patterns. As shown in 3.2, the day of the week and the hour of the day also impact the workload, so these two features are included. The prediction target is the actual load value for that time point. We employ XGBoost [4] as the prediction model due to its simplicity and effectiveness. We compute the normalized mean absolute error (NMAE) and normalized mean square error (NMSE) between the prediction and the ground truth. The results are shown in Figure 17.

As shown, even without careful tuning, the models can predict future system load easily and accurately. The predicted load trends generally align well with the actual load, indicating significant opportunities for fine-grained load prediction modeling in real-world inference services, which lead to more precise service delivery.

We also investigate the performance of the prediction algorithm under different time intervals. As shown in Figure 18, predicting long-term patterns (10 minutes) is generally easier than predicting short-term patterns (1 minute), with the former achieving a 58% lower NMSE compared to the latter. This suggests that when using load prediction for serving, it is essential to balance the granularity of scheduling with the accuracy of predictions.

6 BurstGPT in Industry

We provide an example of how we use `BurstGPT` for industrial use in Huawei during our development of *PD* disaggregation systems. As a common practice in the industry, given a fixed serving system scale, developers use goodput [53] to evaluate the system's capacity, i.e., the maximum request rate that can be served while meeting the SLO goal (e.g., 90%) for each GPU. Recently, disaggregation of prefilling and decoding processes during LLM inference has become the standard approach in LLM service deployment [15, 28, 31, 33, 53]. However, scheduling the disaggregation of prefilling and decoding

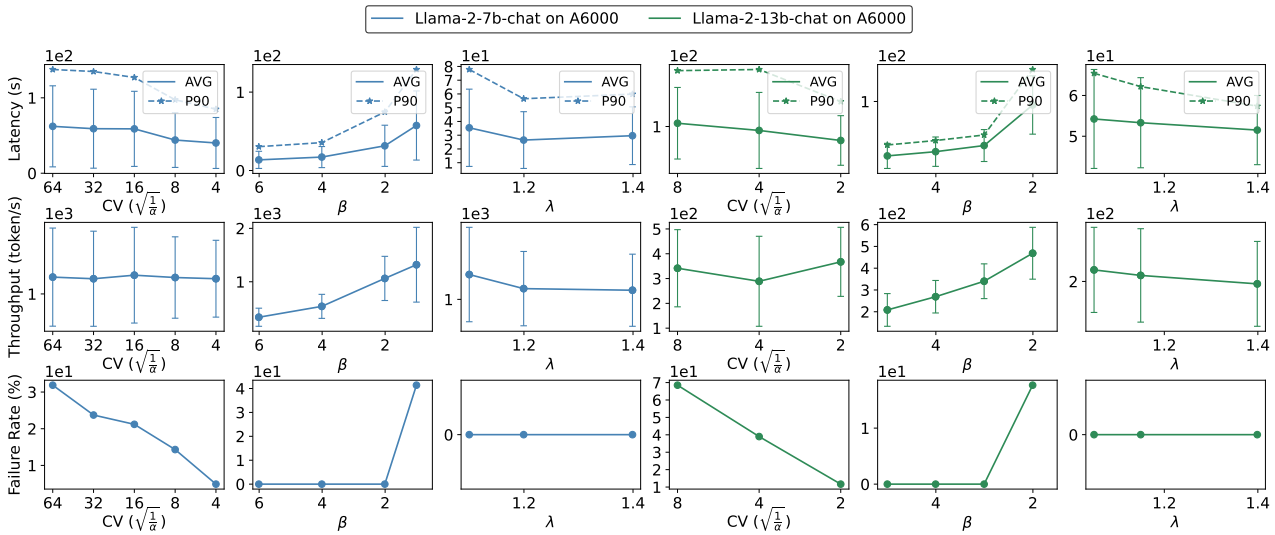


Figure 15: Impact of parameters α, β, and λ on average and p90 latency, average throughput, and average failure rates in vLLM. The evaluation is conducted with Llama-2-7b-chat and Llama-2-13b-chat models using A6000 and A800 GPU Servers. Experiment Settings: Larger α or smaller CV and β increase system burstiness; higher λ correlates with shorter request likelihood. Note: Due to space limits, only results from the A6000 server are shown.

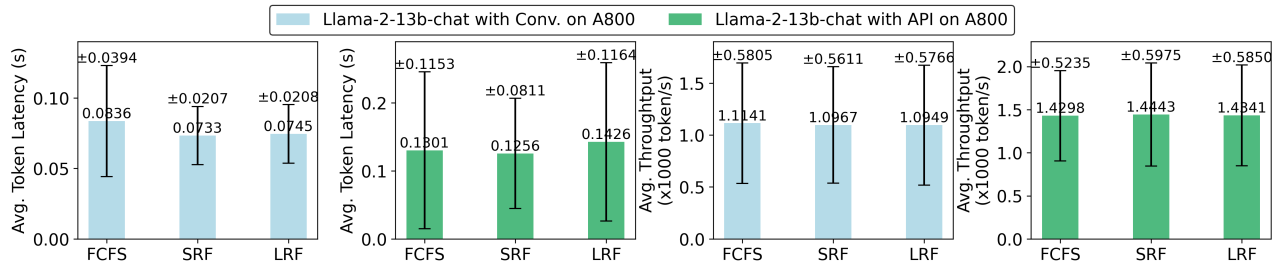


Figure 16: Average token latency and throughput of FCFS, SRF, and LRF on vLLM serving Llama-2-13b-chat on an A800 GPU server with `BurstGPT` Conversations and API workloads.

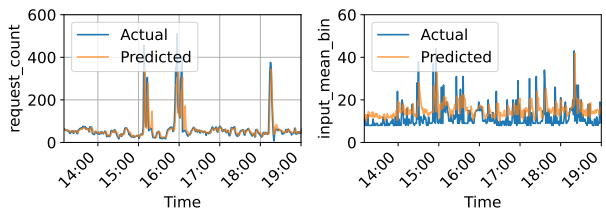


Figure 17: Request count and mean request token number predictions. The figure only shows a random interval of 6 hours. The time bin interval is 1 minute. Left: Request count predictions, the NMAE is 0.73 and the NMSE is 0.48. Right: Mean request token number predictions, the NMAE is 0.72 and the NMSE is 0.67.

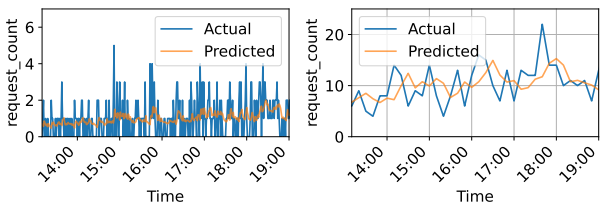


Figure 18: Request count predictions. The figure only shows a random interval of 6 hours. Left: Request count predictions of Conv. with time bin interval 1 minute, the NMAE is 0.66 and the NMSE is 0.50. Right: Request count predictions of Conv. with time bin interval 10 minutes, the NMAE is 0.32, and the NMSE is 0.21.

can be challenging. The prefilling instances can be seen as the "Provider," which generates the first KV cache and token, while the decoding instances act as the "Consumer," accepting the KV cache and token to continue the auto-regressive generation. Thus, for a given system scale (i.e., the number of prefilling (P) and decoding (D) instances), the concurrency of requests, request lengths, and

response lengths will determine the goodput of the system. As workloads vary rapidly in real-world serving, a fixed *PD* ratio is not always ideal. Instead, a dynamic *PD* ratio should be used.

Figure 20 and Figure 19 show examples of goodput at different PD ratios. Since we lack preliminary workload traces before the real-world deployment of our serving system, we use `BurstGPT` in our simulation platform to determine the optimal PD ratio under various user, model, and system workloads. The result is used to develop prototypes of the *PD* disaggregation serving system at Huawei. In the simulation, we select the *P* and *D* instance ratio, then run simulations, gradually increasing QPS while monitoring the delay SLO achievement rate until the SLO is no longer met. The QPS at this point is considered the system's capacity. We then fix the total number of *P* and *D* instances. First, considering a simple situation, we adjust the *PD* ratio once to fit the workload divergence in Figure 19, we observe that at 6:00, the *PD* ratio switches from 2:6 to 6:2, improving the goodput of the serving system. This simulation serves as a solid prototype for real-world *PD* disaggregation scheduling policies. Then, we develop a dynamic *PD* ratio strategy. We use beam search to identify the best *PD* ratio given the provisioned workload in BurstGPT. Instead of directly scaling the number of instances, we simulate scaling by adjusting the load size per instance, assuming a predefined instance launch time in seconds. As shown in Figure 20, with the provisioned workload, the system searches for the optimal *PD* ratio to improve the goodput.

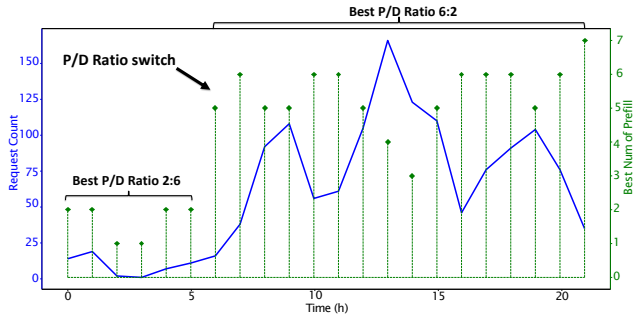


Figure 19: At 6:00, the PD ratio switches from 2:6 to 6:2, improving the goodput of the serving system.

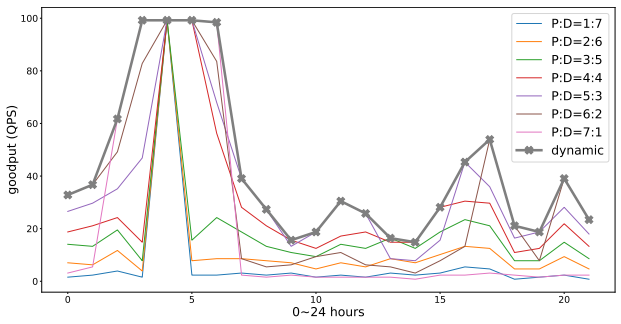


Figure 20: Given 8 instances, comparison of different PD ratios and dynamic PD ratios based on workload provisioning and beam search during LLM serving.

These findings highlight the significant improvements in LLM serving achieved through data-driven modeling of user and model behavior. By carefully designing the scheduling algorithm, we can greatly enhance both system efficiency and QoS.

7 Broader Impacts and Conclusions

This research emphasizes incorporating real-world workload data to enhance LLM serving systems. It identifies a critical gap: the need for such data in optimizing these systems. In this work, we introduce the real-world LLM workload `BurstGPT` from the Azure OpenAI GPT service. The traces and evaluations reveal possible performance degradation of serving systems under real-world workloads, highlighting challenges and opportunities in workload-aware LLM serving optimizations.

We encourage using `BurstGPT` in optimizing and evaluating serving systems to improve the efficiency, stability, and reliability of LLM services under realistic workloads. We also advocate for data-driven methodologies in developing LLM systems in the future.

8 Acknowledgement

This work that involves all hardware use was done and open-sourced while Yuxin Wang was a research intern at the Hong Kong University of Science and Technology (Guangzhou). It was partially supported by the National Natural Science Foundation of China under Grant No. 62272122, a Hong Kong RIF grant under Grant No. R6021-20, and Hong Kong CRF grants under Grant No. C2004-21G and C7004-22G. Also supported by the High-performance Computing Platform and the Red Bird MPhil Program of the Hong Kong University of Science and Technology (Guangzhou). For all closed-source models, evaluations are performed through their respective APIs.

References

[1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming throughput-latency tradeoff in LLM inference with sarathi-serve. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation* (Santa Clara, CA, USA) *(OSDI'24)*. USENIX Association, USA, Article 7, 18 pages.

[2] Muhammad Ahsan, Sacheendra Talluri, and Alexandru Iosup. 2023. Failure Analysis of Big Cloud Service Providers Prior to and During Covid-19 Period. arXiv:2210.08006 [cs.DC]

[3] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245* (2023).

[4] Tianqi Chen and Carlos Guestrin. 2016. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*. 785–794.

[5] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=mZn2Xyh9Ec

[6] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems* 35 (2022), 16344–16359.

[7] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Proceedings of the 36th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '22)*. Curran Associates Inc., Red Hook, NY, USA, Article 2198, 15 pages.

[8] Ruibo Fan, Xiangrui Yu, Peijie Dong, Zeyu Li, Gu Gong, Qiang Wang, Wei Wang, and Xiaowen Chu. 2025. SpInfer: Leveraging Low-Level Sparsity for Efficient Large Language Model Inference on GPUs. In *Proceedings of the Twentieth European Conference on Computer Systems* (Rotterdam, Netherlands) *(EuroSys '25)*. Association for Computing Machinery, New York, NY, USA, 243–260. doi:10.1145/3689031.3717481

[9] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. USENIX Association, Santa Clara, CA, 111–126. https://www.usenix.org/conference/atc24/presentation/gao-bin-cost

[10] Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. 2023. Model Tells You What to Discard: Adaptive KV Cache Compression for LLMs. *arXiv preprint arXiv:2310.01801* (2023).

[11] Ruihao Gong, Shihao Bai, Siyu Wu, Yunqian Fan, Zaijun Wang, Xiuhong Li, Hailong Yang, and Xianglong Liu. 2025. Past-Future Scheduler for LLM Serving under SLA Guarantees. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 798–813.

[12] Arnav Gudibande, Eric Wallace, Charlie Snell, Xinyang Geng, Hao Liu, Pieter Abbeel, Sergey Levine, and Dawn Song. 2023. The False Promise of Imitating Proprietary LLMs. arXiv:2305.15717 [cs.CL]

[13] Xin He, Shunkang Zhang, Yuxin Wang, Haiyan Yin, Zihao Zeng, Shaohuai Shi, Zhenheng Tang, Xiaowen Chu, Ivor Tsang, and Ong Yew Soon. 2024. Expertflow: Optimized expert activation and token allocation for efficient mixture-of-experts inference. *arXiv preprint arXiv:2410.17954* (2024).

[14] Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiuhong Li, Jun Liu, Kangdi Chen, Yuhan Dong, and Yu Wang. 2024. FlashDecoding++: Faster Large Language Model Inference with Asynchronization, Flat GEMM Optimization, and Heuristics. In *Proceedings of Machine Learning and Systems*, P. Gibbons, G. Pekhimenko, and C. De Sa (Eds.), Vol. 6. 148–161. https://proceedings.mlsys.org/paper_files/paper/2024/file/5321b1dabcd2be188d796c21b733e8c7-Paper-Conference.pdf

[15] Cunchen Hu, Heyang Huang, Junhao Hu, Jiang Xu, Xusheng Chen, Tao Xie, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. 2024. MemServe: Context Caching for Disaggregated LLM Serving with Elastic Memory Pool. arXiv:2406.17565 [cs.DC] https://arxiv.org/abs/2406.17565

[16] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Yu, Joey Gonzalez, Hao Zhang, and Ion Stoica. 2023. vLLM: Easy, Fast, and Cheap LLM Serving with PagedAttention. https://vllm.ai/

[17] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.

[18] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. InfiniGen: efficient generative inference of large language models with dynamic KV cache management. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation* (Santa Clara, CA, USA) *(OSDI'24)*. USENIX Association, USA, Article 9, 18 pages.

[19] Zhihui Li, Max Gronke, and Charles Steidel. 2023. ALPACA: A New Semi-Analytic Model for Metal Absorption Lines Emerging from Clumpy Galactic Environments. arXiv:2306.11089 [astro-ph.GA]

[20] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E Gonzalez, et al. 2023. AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving. *arXiv preprint arXiv:2302.11665* (2023).

[21] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. USENIX Association, Boston, MA, 663–679. https://www.usenix.org/conference/osdi23/presentation/li-zhouhan

[22] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. 2024. Parrot: efficient serving of LLM-based applications with semantic variable. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation* (Santa Clara, CA, USA) *(OSDI'24)*. USENIX Association, USA, Article 50, 17 pages.

[23] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Hongyi Jin, Tianqi Chen, and Zhihao Jia. 2023. Towards Efficient Generative Large Language Model Serving: A Survey from Algorithms to Systems. *arXiv preprint arXiv:2312.15234* (2023).

[24] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. 2024. SpecInfer: Accelerating Large Language Model Serving with Tree-based Speculative Inference and Verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 932–949. doi:10.1145/3620666.3651335

[25] Xupeng Miao, Chunan Shi, Jiangfei Duan, Xiaoli Xi, Dahua Lin, Bin Cui, and Zhihao Jia. 2024. SpotServe: Serving Generative Large Language Models on Preemptible Instances. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 1112–1127. doi:10.1145/3620665.3640411

[26] Microsoft. 2024. Azure OpenAI Service. https://azure.microsoft.com/en-us/products/ai-services/openai-service/

[27] Oh Neal, Vaidya anf Fred and Comly Nick. 2023. Optimizing inference on large language models with nvidia tensorrt-llm. https://github.com/NVIDIA/TensorRT-LLM

[28] Hyungjun Oh, Kihong Kim, Jaemin Kim, Sungkyun Kim, Junyeol Lee, Du-seong Chang, and Jiwon Seo. 2024. ExeGPT: Constraint-Aware Resource Scheduling for LLM Inference. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 369–384. doi:10.1145/3620665.3640383

[29] OpenAI. 2022. Introducing ChatGPT. https://openai.com/blog/chatgpt

[30] OpenAI. 2023. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774* (2023).

[31] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 118–132. doi:10.1109/ISCA59077.2024.00019

[32] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. 2025. vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (Rotterdam, Netherlands) *(ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 1133–1150. doi:10.1145/3669940.3707256

[33] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. USENIX Association, Santa Clara, CA, 155–170. https://www.usenix.org/conference/fast25/presentation/qin

[34] Jeff Rasley and Rajbhandari et.al. 2020. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 3505–3506.

[35] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. https://openreview.net/forum?id=tVConYid20

[36] Mohammad Shahrad, Rodrigo Fonseca, Inigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. 2020. Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, 205–218. https://www.usenix.org/conference/atc20/presentation/shahrad

[37] Noam Shazeer. 2019. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150* (2019).

[38] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E. Gonzalez, and Ion Stoica. 2024. Fairness in Serving Large Language Models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 965–988. https://www.usenix.org/conference/osdi24/presentation/sheng

[39] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: dynamic scheduling for large language model serving. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation* (Santa Clara, CA, USA) *(OSDI'24)*. USENIX Association, USA, Article 10, 19 pages.

[40] Zhenheng Tang, Yonggang Zhang, Peijie Dong, Yiu ming Cheung, Amelie Chi Zhou, Bo Han, and Xiaowen Chu. 2024. FuseFL: One-Shot Federated Learning through the Lens of Causality with Progressive Model Fusion. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. https://openreview.net/forum?id=E7fZOoiEKl

[41] Gemini Team, Machel Reid, Nikolay Savinov, and etc. 2024. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. arXiv:2403.05530 [cs.CL]

[42] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971 [cs.CL]

[43] Bingyang Wu, Yinmin Zhong, Zili Zhang, Gang Huang, Xuanzhe Liu, and Xin Jin. 2023. Fast Distributed Inference Serving for Large Language Models. arXiv:2305.05920 [cs.LG]

[44] Chenpeng Wu, Qiqi Gu, Heng Shi, Jianguo Yao, and Haibing Guan. 2025. Samoyeds: Accelerating MoE Models with Structured Sparsity Leveraging Sparse Tensor Cores. In *Proceedings of the Twentieth European Conference on Computer*

Systems (Rotterdam, Netherlands) *(EuroSys '25)*. Association for Computing Machinery, New York, NY, USA, 293–310. doi:10.1145/3689031.3717455

[45] Haojun Xia, Zhen Zheng, Yuchao Li, Donglin Zhuang, Zhongzhu Zhou, Xiafei Qiu, Yong Li, Wei Lin, and Shuaiwen Leon Song. 2023. Flash-LLM: Enabling Cost-Effective and Highly-Efficient Large Generative Model Inference with Unstructured Sparsity. *Proc. VLDB Endow.* 17, 2 (Oct. 2023), 211–224. doi:10.14778/3626292.3626303

[46] Haojun Xia, Zhen Zheng, Xiaoxia Wu, Shiyang Chen, Zhewei Yao, Stephen Youn, Arash Bakhtiari, Michael Wyatt, Donglin Zhuang, Zhongzhu Zhou, Olatunji Ruwase, Yuxiong He, and Shuaiwen Leon Song. 2024. Quant-LLM: Accelerating the Serving of Large Language Models via FP6-Centric Algorithm-System Co-Design on Modern GPUs. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. USENIX Association, Santa Clara, CA, 699–713. https://www.usenix.org/conference/atc24/presentation/xia

[47] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*. PMLR, 38087–38099.

[48] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In *Proceedings of the Twentieth European Conference on Computer Systems* (Rotterdam, Netherlands) *(EuroSys '25)*. Association for Computing Machinery, New York, NY, USA, 94–109. doi:10.1145/3689031.3696098

[49] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, Carlsbad, CA, 521–538. https://www.usenix.org/conference/osdi22/presentation/yu

[50] Yanqi Zhang, Íñigo Goiri, Gohar Irfan Chaudhry, Rodrigo Fonseca, Sameh Elnikety, Christina Delimitrou, and Ricardo Bianchini. 2021. Faster and Cheaper Serverless Computing on Harvested Resources. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*. doi:10.1145/3477132.3483580

[51] Yilong Zhao, Chien-Yu Lin, Kan Zhu, Zihao Ye, Lequn Chen, Size Zheng, Luis Ceze, Arvind Krishnamurthy, Tianqi Chen, and Baris Kasikci. 2024. Atom: Low-Bit Quantization for Efficient and Accurate LLM Serving. In *Proceedings of Machine Learning and Systems*, P. Gibbons, G. Pekhimenko, and C. De Sa (Eds.), Vol. 6. 196–209. https://proceedings.mlsys.org/paper_files/paper/2024/file/5edb57c05c81d04beb716ef1d542fe9e-Paper-Conference.pdf

[52] Zangwei Zheng, Xiaozhe Ren, Fuzhao Xue, Yang Luo, Xin Jiang, and Yang You. 2023. Response length perception and sequence scheduling: an LLM-empowered LLM inference pipeline. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 2859, 14 pages.

[53] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation* (Santa Clara, CA, USA) *(OSDI'24)*. USENIX Association, USA, Article 11, 18 pages.

[54] Jiahang Zhou, Yanyu Chen, Zicong Hong, Wuhui Chen, Yue Yu, Tao Zhang, Hui Wang, Chuanfu Zhang, and Zibin Zheng. 2024. Training and Serving System of Foundation Models: A Comprehensive Survey. arXiv:2401.02643 [cs.AI]